

AI Foundations & Exam Context

Chapter 01 · Episode 01 · Domain 1.1 — Why AI Changes Security



Learning Objectives

1. Define AI and explain probabilistic vs deterministic systems
2. Identify the MDLC phases and their security significance
3. Map the four SecAI+ exam domains and how they build

The Most Important Concept in This Course

DETERMINISTIC SOFTWARE

Same input → same output, every time

Trace the code, prove behavior

Test your way to confidence

e.g. firewall rule, hash, encryption

PROBABILISTIC AI

Same prompt → possibly different output

Learns statistical patterns from data

No code path, no if-statement to audit

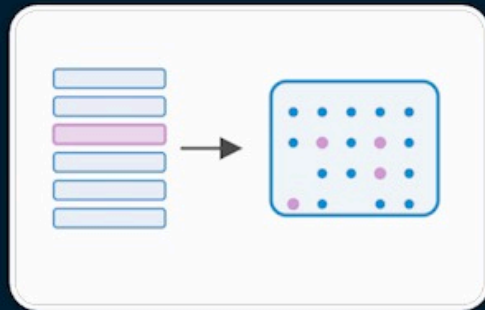
Changes how you must secure it

You Can't Test Your Way to a Guarantee

Unit tests can't prove a security guarantee
Treat the model as an opaque probabilistic component
Design your controls around that reality

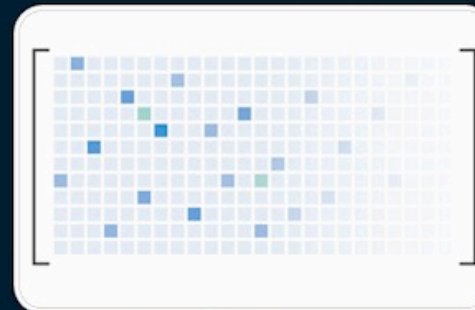


Core Vocabulary the Exam Expects



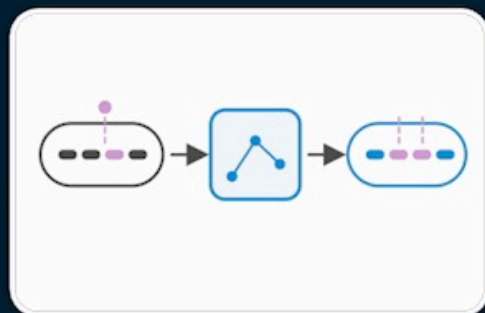
TRAINING

Build model from data (pre-deploy); poison here = compromised



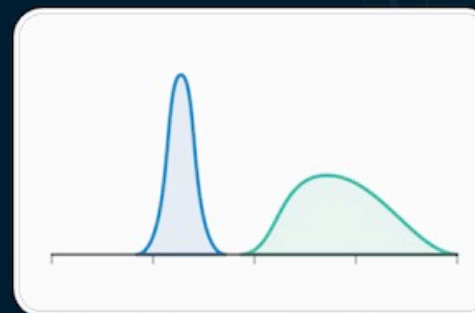
PARAMETERS

Opaque learned weights — billions, unreadable



INFERENCE

Using the model; where injection & jailbreak live



TEMPERATURE

Output randomness; high = higher variance

Context Window = The Injection Surface

Max tokens the model processes in
one interaction

Anything in context can influence the output

The foundation of prompt injection

MDLC = the end-to-end AI lifecycle
(tested throughout)



Four Exam Domains — Your Roadmap



DOMAIN 1 — 17%

Basic AI concepts
for cybersecurity



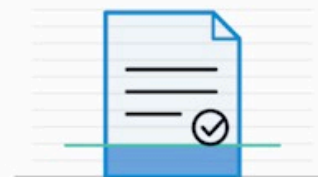
DOMAIN 3 — 24%

AI-assisted security:
tool & weapon



DOMAIN 2 — 40%

Securing AI systems (largest)



DOMAIN 4 — 19%

AI governance, risk & compliance

The Mindset Shift

TRADITIONAL SECURITY

Find the vulnerability

Patch the code

Verify the fix

Deterministic, repeatable, auditable

AI SECURITY

System may behave unexpectedly

...even when operating normally

Confidence, not certainty

Design for opacity & variability

The AI Security Mindset — 5 Principles

- 1 — Attack surface includes training data, not just code
- 2 — Behavior can't be fully verified by testing
- 3 — Training-time attacks surface at inference time
- 4 — Outputs can be manipulated without touching the model
- 5 — Governance must be continuous, not point-in-time

1 SAME
Deterministic Hash

2 VARIES
Probabilistic Model

3 COMPARE
Temperature Effect

HOW TO RUN

WINDOWS POWERSHELL

```
python deterministic_vs_probabilistic.py
```

MACOS / LINUX

```
python3 deterministic_vs_probabilistic.py
```

TRY IT Change the temperature value (0.1 → 0.9) and observe how output variance changes.

`deterministic_vs_probabilistic.py`

pure Python 3.10+, no external libraries

Demo: Deterministic vs Probabilistic

SHA-256 hash (same output) vs a simulated model (varies) + temperature effect

The MDLC — Seven Phases

- 1 — Use-case definition
- 2 — Data collection
- 3 — Model development
- 4 — TEVV validation ★ — the formal security/quality gate before deploy
- 5 — Deployment
- 6 — Monitoring & maintenance
- 7 — Feedback & iteration

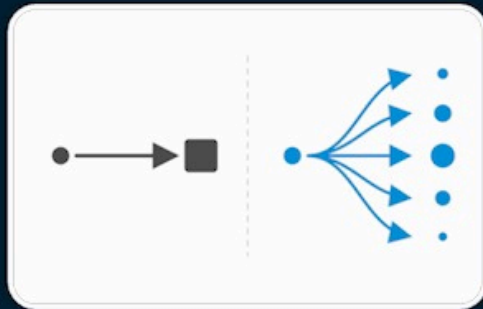
Fail TEVV → it goes back. This gate is tested directly.

Security Is Continuous Across the MDLC

Poisoned phase-2 data passes every deployment check
Skipping early phases multiplies remediation cost
Every Domain 2 control maps to an MDLC phase
Ask: which phase does this control protect?



Exam Tips



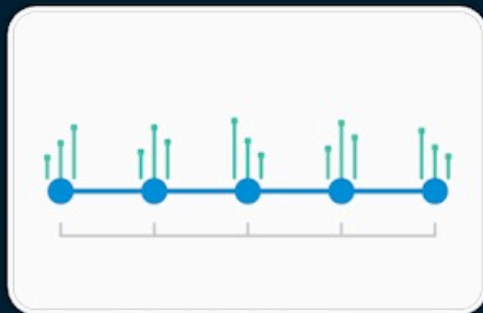
AI IS PROBABILISTIC

Deterministic assumptions
don't apply



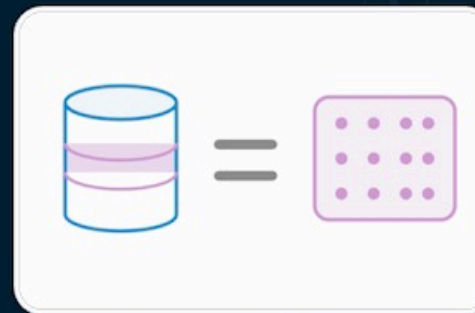
4 DOMAINS IN ORDER

Concepts → secure →
use → govern



MDLC IS THE BACKBONE

Every D2 control maps
to a phase



DATA = ATTACK SURFACE

Data compromise = model
compromise

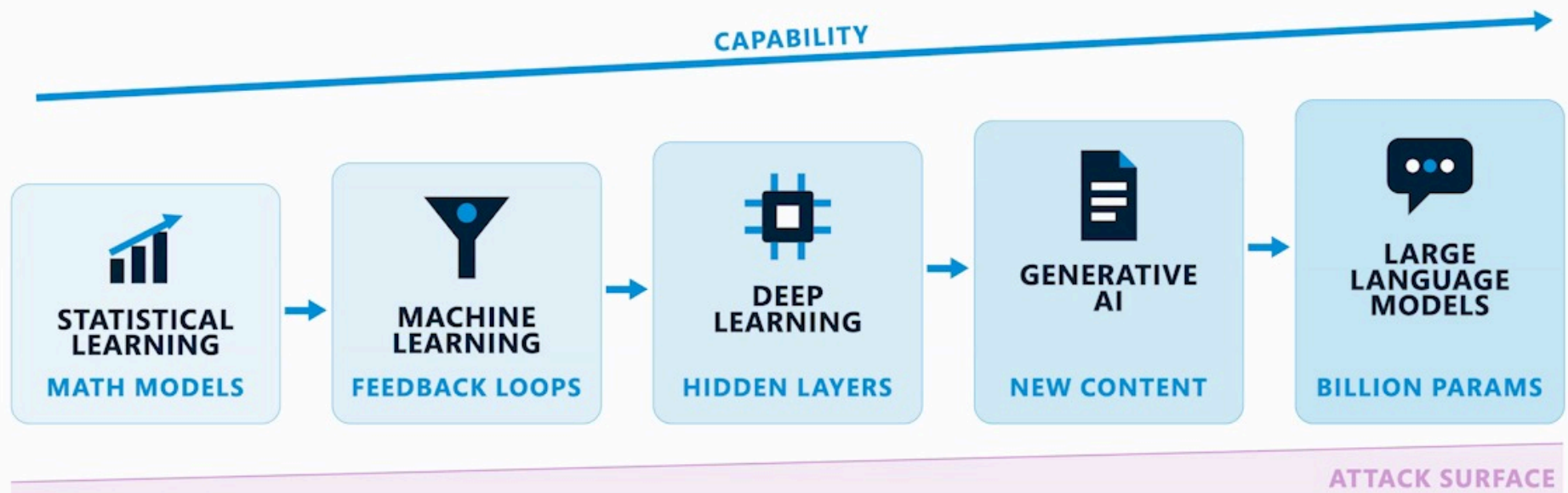
AI Types: Generative, ML, Statistical & Deep Learning

Chapter 01 · Episode 02 · Domain 1.1 — The AI Type Spectrum



Learning Objectives

1. Compare AI type architectures and how each differs
2. Map each AI type to its cybersecurity application
3. Identify the primary attack surface for each type



The AI Type Spectrum — Capability & Attack Surface →

From pattern matching to content generation, each AI type introduces new capabilities and new attack surfaces.

Data-Layer

STATISTICAL LEARNING

Regression, clustering, Bayesian

Baseline anomaly detection, fraud scoring

Attack: corrupt the baseline data

MACHINE LEARNING

Learns from experience & feedback

Malware classifiers, spam, EDR

Attack: label poisoning + pipeline

Deep Learning — Three Tested Threats



INVERSION

Model inversion — query
Reconstruct training data (PII)



ADVERSARIAL

Crafted inputs misclassify
Malware bypasses the classifier



OPACITY

Explainability gap — can't audit
Hidden bias surfaces in production

Generative AI — Bidirectional Impact

DEFENDER APPLICATIONS

Threat-intel summarization
IR documentation drafting
Code review & vuln analysis
Detection-rule generation

ATTACKER APPLICATIONS

Phishing content at scale
Deepfake audio/video
Malware variant generation
Adversarial inputs vs classifiers

LLM vs SLM — Deployment Tradeoffs

LLM

Billions—hundreds of billions parameters

Cloud API — data leaves the org

Broad capability, strong reasoning

Risk: cloud data exposure

SLM

Millions—a few billion parameters

On-premises / edge — data stays internal

Narrower, easier to fine-tune

Risk: validation gap + model theft

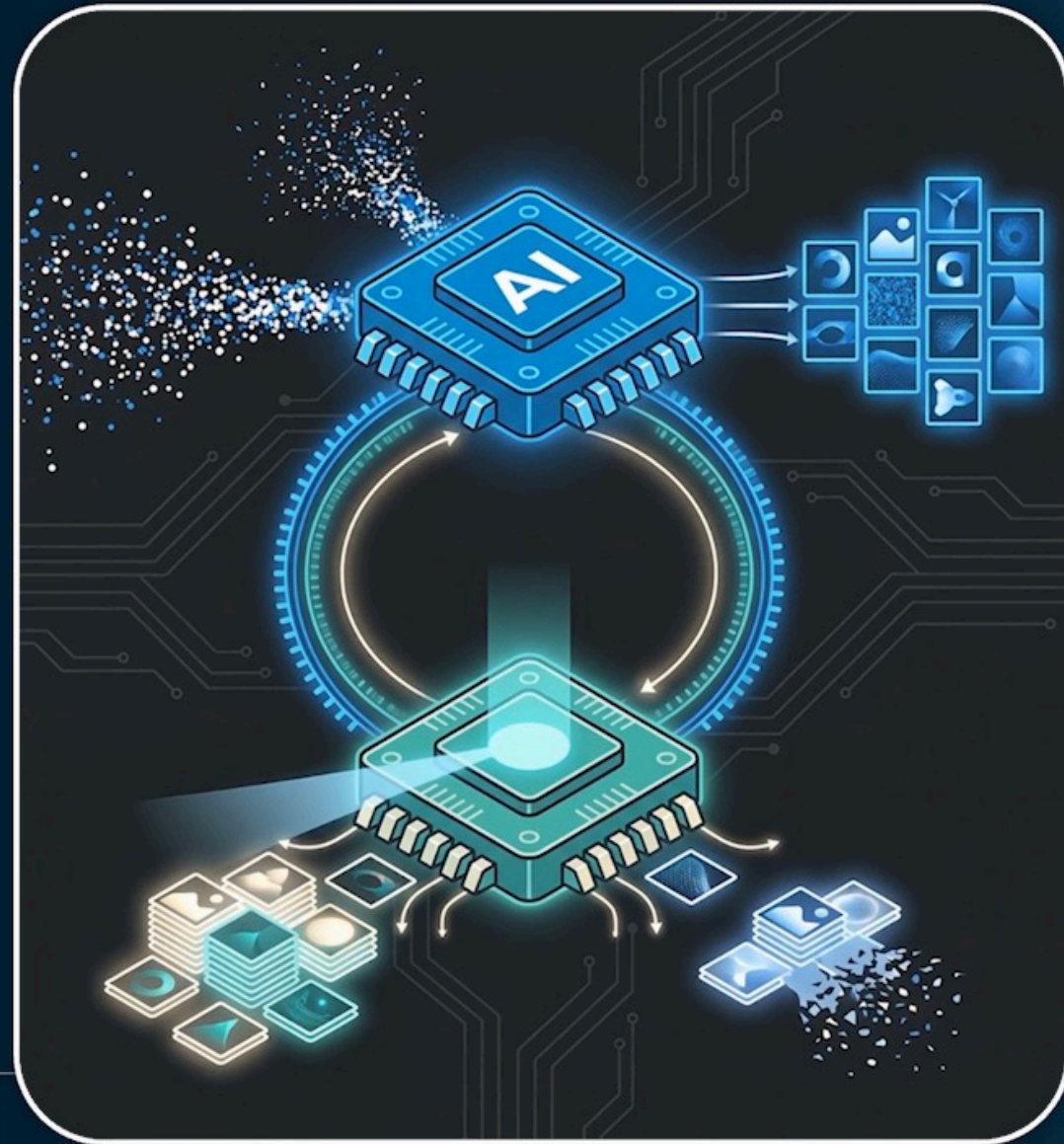
GANs — Two Models Compete

Generator: random noise → synthetic output

Discriminator: sorts real from fake

Generator improves on each iteration

Loops until it fools the discriminator



GAN Security Applications



DEEPPFAKES

Impersonation, CEO fraud, synthetic identity



ADVERSARIAL

Fool classifiers — malware evasion



POISONING

Inject GAN samples into training data

AI Type → Attack Matrix (Tested Directly)



STAT + ML

Data-layer: baseline
& label poisoning



GEN AI / LLMs

Interface: prompt
injection & jailbreak



DEEP LEARNING

Model-level:
inversion + adversarial



GANs

Synthetic content: deepfakes

The Match Rule

Questions go both directions: attack $\rightarrow$ type
or type $\rightarrow$ attack

Match attack technique $\rightarrow$ AI type $\rightarrow$ correct control

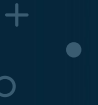
That chain answers the question

Group by layer: data · model · interface · synthetic



Transformers, LLMs, SLMs & GANs

Chapter 01 · Episode 03 · Domain 1.1 — Architecture Deep-Dive





Learning Objectives

1. Explain the transformer mechanism and attention
2. Contrast LLM vs SLM deployment security tradeoffs
3. Describe GAN adversarial training and its offensive uses

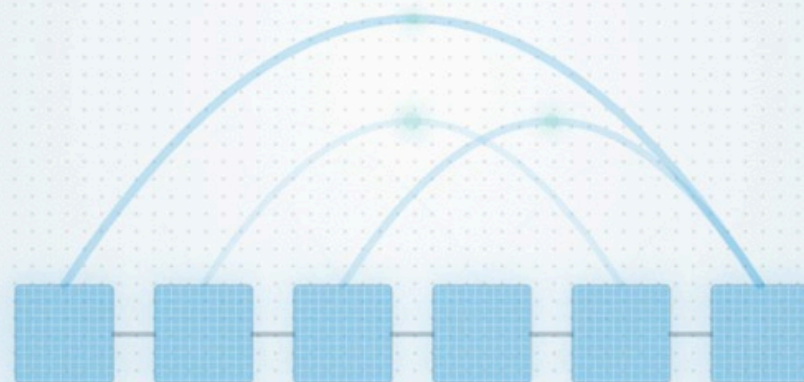
Attention — Parallel, Not Sequential

Pre-2017 models read text word-by-word (slow)

Attention weighs every token against every other

Processes all tokens at once → scales with compute

Handles long context; doesn't forget like sequential models



Context Window = The Attack Surface

Everything is tokenized (word fragments)

Context window = the model's working memory

Anything in it can influence the output

Injection via a document, web page, or tool result



LLM vs SLM — Security Deployment

LLM

Billions—hundreds of billions parameters

Cloud API — data leaves your network

GPT, Claude, Gemini

Risk: cloud data exposure

SLM

Millions—a few billion parameters

On-premises / edge — data stays internal

Faster, cheaper, more fine-tuning

Risk: validation gap (less scrutiny)

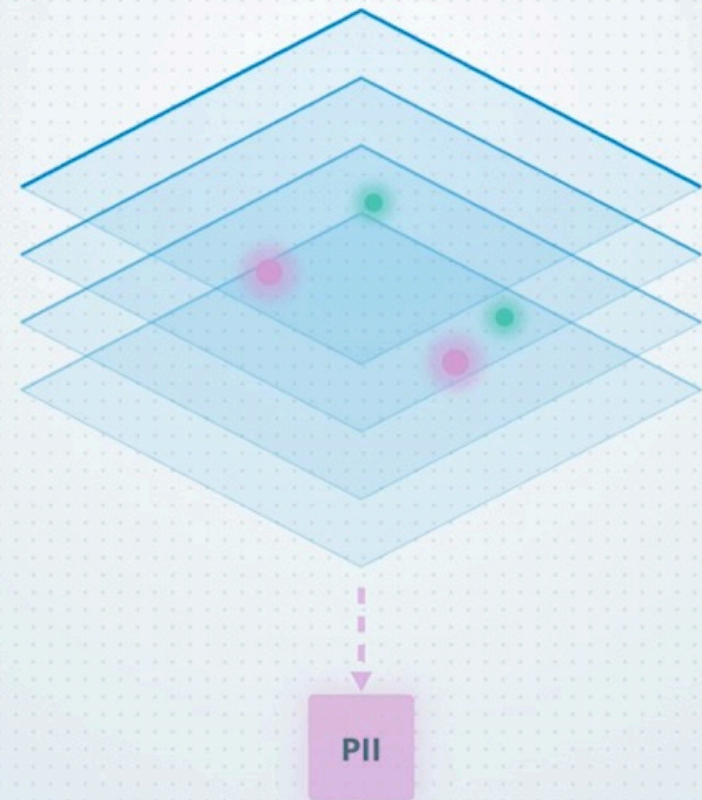
Fine-Tuning Encodes Your Data

Fine-tuning adapts a base model on your data

That sensitive data becomes part of the weights

Model inversion attempts to extract it

Model theft / over-querying can recover it



GANs — Adversarial Training

Generator: random noise → synthetic output

Discriminator: real vs fake — tries not to be fooled

Both improve in opposition each round

Offense: deepfakes, voice fraud, adversarial images, poisoning



1 Token-by-token generation loop

P
A
R
T

2 Temperature effect on variance

P
A
R
T

3 GAN training round simulation

P
A
R
T

HOW TO RUN

WINDOWS

```
python token_prediction_simulator.py
```

MACOS/LINUX

```
python3 token_prediction_simulator.py
```

TRY IT Adjust temperature in Part 2, then observe how the GAN's discriminator accuracy changes over training rounds in Part 3.

```
token_prediction_simulator.py
```

pure Python 3.10+, no external libraries

Demo: Token Prediction & GAN Simulator

Token-by-token generation · temperature vs variance · RAG injection · a GAN training round

Three Architecture-Level Security Principles



CONTEXT

= the attack surface

Validate, sanitize, audit inputs



SCALE

= opacity

Monitor continuously
to compensate



ADVERSARIES

GANs generate them

Test classifiers in TEVV

Exam Tips



ATTENTION

Parallel context processing



LLM vs SLM

Cloud exposure vs
validation gap



FINE-TUNING

Encodes sensitive data (inversion)



GANs

Deepfakes + adversarial examples